# Import Required Libraries

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

# Load the Dataset

In [2]:

```
df = pd.read_csv("WA_Fn-UseC_-HR-Employee-Attrition.csv")
```

In [3]:

```
df.head()
```

Out[3]:

| | Age | Attrition | BusinessTravel | DailyRate | Department |
|---|---|---|---|---|---|
| **0** | 41 | Yes | Travel_Rarely | 1102 | Sales |
| **1** | 49 | No | Travel_Frequently | 279 | Research & Development |
| **2** | 37 | Yes | Travel_Rarely | 1373 | Research & Development |
| **3** | 33 | No | Travel_Frequently | 1392 | Research & Development |
| **4** | 27 | No | Travel_Rarely | 591 | Research & Development |

5 rows × 35 columns

# Basic Data Understanding

In [4]:

```
df.shape # rows, columns
```

Out[4]:

```
(1470, 35)
```

In [5]:

```
df.info() # data types
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count
Dtype
---  ------                    --------------
-----
 0   Age                       1470 non-null
int64
 1   Attrition                 1470 non-null
object
 2   BusinessTravel            1470 non-null
object
 3   DailyRate                 1470 non-null
int64
 4   Department                1470 non-null
object
 5   DistanceFromHome          1470 non-null
int64
 6   Education                 1470 non-null
int64
 7   EducationField            1470 non-null
object
 8   EmployeeCount             1470 non-null
int64
 9   EmployeeNumber            1470 non-null
int64
 10  EnvironmentSatisfaction   1470 non-null
int64
 11  Gender                    1470 non-null
object
 12  HourlyRate                1470 non-null
int64
 13  JobInvolvement            1470 non-null
int64
 14  JobLevel                  1470 non-null
int64
 15  JobRole                   1470 non-null
object
 16  JobSatisfaction           1470 non-null
int64
```

```
 17  MaritalStatus             1470 non-null
object
 18  MonthlyIncome             1470 non-null
int64
 19  MonthlyRate               1470 non-null
int64
 20  NumCompaniesWorked        1470 non-null
int64
 21  Over18                    1470 non-null
object
 22  OverTime                  1470 non-null
object
 23  PercentSalaryHike         1470 non-null
int64
 24  PerformanceRating         1470 non-null
int64
 25  RelationshipSatisfaction  1470 non-null
int64
 26  StandardHours             1470 non-null
int64
 27  StockOptionLevel          1470 non-null
int64
 28  TotalWorkingYears         1470 non-null
int64
 29  TrainingTimesLastYear     1470 non-null
int64
 30  WorkLifeBalance           1470 non-null
int64
 31  YearsAtCompany            1470 non-null
int64
 32  YearsInCurrentRole        1470 non-null
int64
 33  YearsSinceLastPromotion   1470 non-null
int64
 34  YearsWithCurrManager      1470 non-null
int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

In [6]:

```
df.describe() # Statistical Summary
```

Out[6]:

| | Age | DailyRate | DistanceFromHome | Ec |
|---|---|---|---|---|
| **count** | 1470.000000 | 1470.000000 | 1470.000000 | 1470 |
| **mean** | 36.923810 | 802.485714 | 9.192517 | 2 |
| **std** | 9.135373 | 403.509100 | 8.106864 | 1 |
| **min** | 18.000000 | 102.000000 | 1.000000 | 1 |
| **25%** | 30.000000 | 465.000000 | 2.000000 | 2 |
| **50%** | 36.000000 | 802.000000 | 7.000000 | 3 |
| **75%** | 43.000000 | 1157.000000 | 14.000000 | 4 |
| **max** | 60.000000 | 1499.000000 | 29.000000 | 5 |

8 rows × 26 columns

In [7]:

```
df.columns # Columns names
```

Out[7]:

```
Index(['Age', 'Attrition', 'BusinessTravel',
'DailyRate', 'Department',
       'DistanceFromHome', 'Education', 'Educa
tionField', 'EmployeeCount',
       'EmployeeNumber', 'EnvironmentSatisfact
ion', 'Gender', 'HourlyRate',
       'JobInvolvement', 'JobLevel', 'JobRol
e', 'JobSatisfaction',
       'MaritalStatus', 'MonthlyIncome', 'Mont
hlyRate', 'NumCompaniesWorked',
       'Over18', 'OverTime', 'PercentSalaryHik
e', 'PerformanceRating',
       'RelationshipSatisfaction', 'StandardHo
urs', 'StockOptionLevel',
       'TotalWorkingYears', 'TrainingTimesLast
Year', 'WorkLifeBalance',
       'YearsAtCompany', 'YearsInCurrentRole',
'YearsSinceLastPromotion',
       'YearsWithCurrManager'],
      dtype='object')
```

# Data Cleaning

In [8]:

```
df.isnull().sum() # Check Missing Values
```

Out[8]:

```
Age                         0
Attrition                   0
BusinessTravel              0
DailyRate                   0
Department                  0
DistanceFromHome            0
Education                   0
EducationField              0
EmployeeCount               0
EmployeeNumber              0
EnvironmentSatisfaction     0
Gender                      0
HourlyRate                  0
JobInvolvement              0
JobLevel                    0
JobRole                     0
JobSatisfaction             0
MaritalStatus               0
MonthlyIncome               0
MonthlyRate                 0
NumCompaniesWorked          0
Over18                      0
OverTime                    0
PercentSalaryHike           0
PerformanceRating           0
RelationshipSatisfaction    0
StandardHours               0
StockOptionLevel            0
TotalWorkingYears           0
TrainingTimesLastYear       0
WorkLifeBalance             0
YearsAtCompany              0
YearsInCurrentRole          0
YearsSinceLastPromotion     0
YearsWithCurrManager        0
dtype: int64
```

In [9]:

```
df.duplicated().sum() # Check duplicates values
```

Out[9]:

```
0
```

In [10]:

```
# Drop Irrelevant Columns
df.drop(columns = ['EmployeeCount', 'StandardHours', 'Over18
```

In [11]:

```
df.head()
```

Out[11]:

| | Age | Attrition | BusinessTravel | DailyRate | Department |
|---|---|---|---|---|---|
| **0** | 41 | Yes | Travel_Rarely | 1102 | Sales |
| **1** | 49 | No | Travel_Frequently | 279 | Research & Development |
| **2** | 37 | Yes | Travel_Rarely | 1373 | Research & Development |
| **3** | 33 | No | Travel_Frequently | 1392 | Research & Development |
| **4** | 27 | No | Travel_Rarely | 591 | Research & Development |

5 rows × 32 columns

In [12]:

```
# Fix Data Types
df['Attrition'] = df['Attrition'].map({'Yes':1, 'No':0})
```

# Feature Engineering

In [13]:

```
# Create Age Group
df['AgeGroup'] = pd.cut(df['Age'], bins = [18,25,35,45,55,65]
```

In [14]:

```
df.head()
```

Out[14]:

| | Age | Attrition | BusinessTravel | DailyRate | Department |
|---|---|---|---|---|---|
| **0** | 41 | 1 | Travel_Rarely | 1102 | Sales |
| **1** | 49 | 0 | Travel_Frequently | 279 | Research & Development |
| **2** | 37 | 1 | Travel_Rarely | 1373 | Research & Development |
| **3** | 33 | 0 | Travel_Frequently | 1392 | Research & Development |
| **4** | 27 | 0 | Travel_Rarely | 591 | Research & Development |

5 rows × 33 columns

In [15]:

```
# Create Tensure Group
df['TenureGroup'] = pd.cut(df['YearsAtCompany'], bins = [0,2,
```

In [16]:

```
df.head()
```

Out[16]:

| | Age | Attrition | BusinessTravel | DailyRate | Department |
|---|---|---|---|---|---|
| **0** | 41 | 1 | Travel_Rarely | 1102 | Sales |
| **1** | 49 | 0 | Travel_Frequently | 279 | Research & Development |
| **2** | 37 | 1 | Travel_Rarely | 1373 | Research & Development |
| **3** | 33 | 0 | Travel_Frequently | 1392 | Research & Development |
| **4** | 27 | 0 | Travel_Rarely | 591 | Research & Development |

5 rows × 34 columns

# Exploratory Data Analysis (EDA)

In [17]:

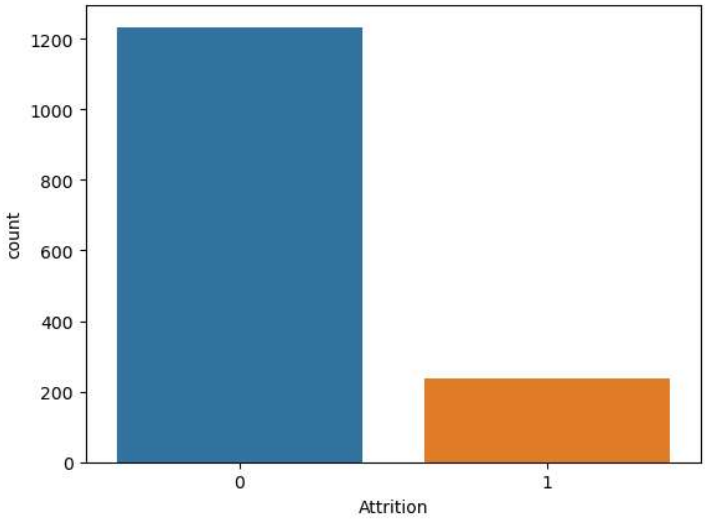
```
# Attrition Distribution

df['Attrition'].value_counts()
```

Out[17]:

```
Attrition
0    1233
1     237
Name: count, dtype: int64
```

In [18]:

```
sns.countplot(x='Attrition', data= df)
plt.show()
```



In [19]:

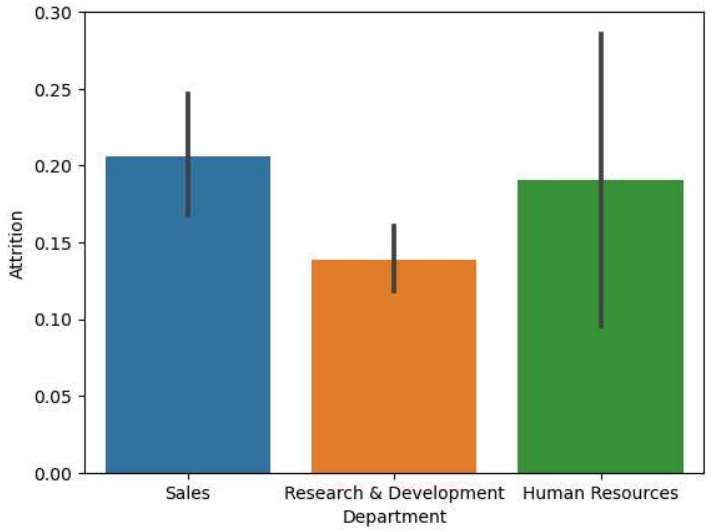
```
# Attrition by Department

df.groupby('Department')['Attrition'].mean()
```

Out[19]:

```
Department
Human Resources           0.190476
Research & Development    0.138398
Sales                     0.206278
Name: Attrition, dtype: float64
```

In [20]:

```
sns.barplot(x='Department', y='Attrition', data = df)
plt.show()
```



In [21]:

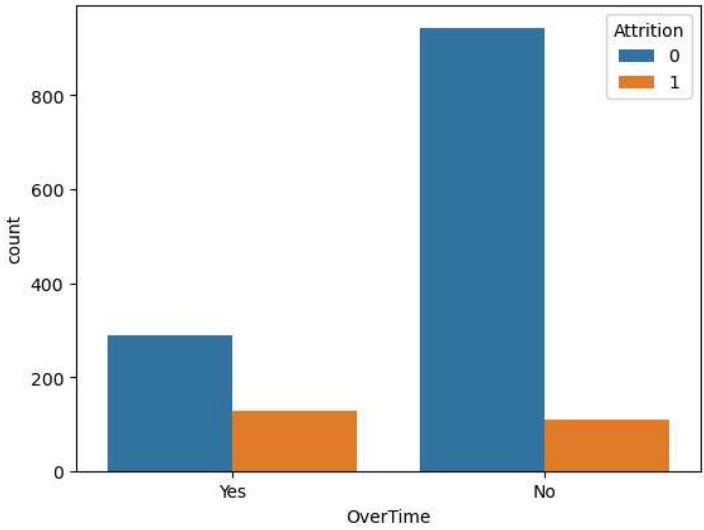
```
# Attrition vs Overtime

pd.crosstab(df['OverTime'], df['Attrition'], normalize = 'inc
```

Out[21]:

| Attrition | 0 | 1 |
|---|---|---|
| **OverTime** | | |
| **No** | 0.895636 | 0.104364 |
| **Yes** | 0.694712 | 0.305288 |

In [22]:

```
sns.countplot(x='OverTime', hue='Attrition', data = df)
plt.show()
```



In [23]:

```
pd.crosstab(df['MonthlyIncome'], df['Attrition'], normalize=
```

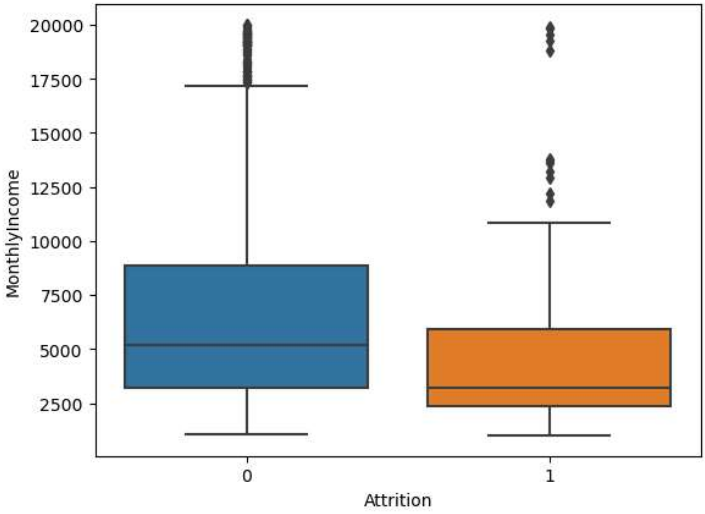
Out[23]:

| Attrition | 0 | 1 |
|---|---|---|
| **MonthlyIncome** | | |
| **1009** | 0.0 | 1.0 |
| **1051** | 1.0 | 0.0 |
| **1052** | 1.0 | 0.0 |
| **1081** | 0.0 | 1.0 |
| **1091** | 0.0 | 1.0 |
| **...** | ... | ... |
| **19859** | 0.0 | 1.0 |
| **19926** | 1.0 | 0.0 |
| **19943** | 1.0 | 0.0 |
| **19973** | 1.0 | 0.0 |
| **19999** | 1.0 | 0.0 |

1349 rows × 2 columns

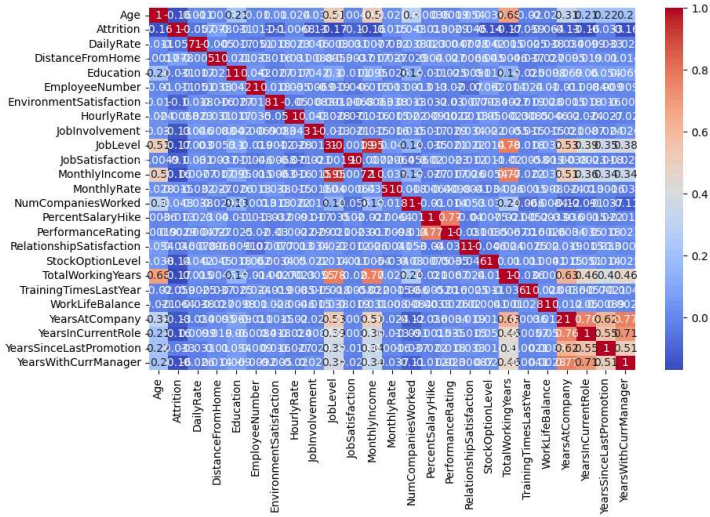
In [24]:

```
# Salary VS Attrition

sns.boxplot(x='Attrition', y='MonthlyIncome', data= df)
plt.show()
```



In [25]:

```
# Correlation Analysis

corr= df.corr(numeric_only= True) # calculation between all r
plt.figure(figsize=(10,6)) # Create figure size width= 10, he
sns.heatmap(corr, annot= True, cmap='coolwarm') # Visualizati
                                                # annot = True: s
                                                # Red: Positive c
                                                # White: Neutral

plt.show()
```




# Advanced Analysis

In [26]:

```
# Attrition Rate
attrition_rate= df['Attrition'].mean()*100
```

In [27]:

```
# High Risk Employees

high_risk = df[(df['OverTime'] == 'Yes') & (df['JobSatisfacti
```

In [28]:

```
# Group Analysis

df.groupby('JobRole').agg({'Attrition':'mean', 'MonthlyIncome
```

Out[28]:

| JobRole | Attrition | MonthlyIncome |
|---|---|---|
| Sales Representative | 0.397590 | 2626.000000 |
| Laboratory Technician | 0.239382 | 3237.169884 |
| Human Resources | 0.230769 | 4235.750000 |
| Sales Executive | 0.174847 | 6924.279141 |
| Research Scientist | 0.160959 | 3239.972603 |
| Manufacturing Director | 0.068966 | 7295.137931 |
| Healthcare Representative | 0.068702 | 7528.763359 |
| Manager | 0.049020 | 17181.676471 |
| Research Director | 0.025000 | 16033.550000 |

# Machine Learning Prediction Model

# Predict whether an employee will leave (Attrition=1) or stay(0)

In [29]:

```
# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScale
from sklearn.metrics import accuracy_score, classification_re

# Models
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
```

# One Hot Encoding

In [30]:

```
df = pd.get_dummies(df, drop_first=True)

# Converts all categorical columns into dummy variables
# drop_first= True: Avoids dummy variable trap (multicollinea
# Reduces redundant columns
```

# Define Features & Target

In [31]:

```
X = df.drop('Attrition', axis=1)
y= df['Attrition']
```

# Train-Test Split

In [32]:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, tes
```

# Feature Scaling

In [33]:

```
scaler= StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

# Train Models

In [34]:

```
# Logistic Regression

lr = LogisticRegression()
lr.fit(X_train, y_train)
y_pred_lr = lr.predict(X_test)
```

In [35]:

```
# Random Forest

rf = RandomForestClassifier(n_estimators=100, random_state=42
rf.fit(X_train, y_train)
y_pred_rf = rf.predict(X_test)
```

# Model Evaluation

In [36]:

```
# Accuracy

print("Logistics Regression Accuracy:", accuracy_score(y_test
print("Random Forest Accuracy:", accuracy_score(y_test, y_pre
```

```
Logistics Regression Accuracy: 0.8979591836734
694
Random Forest Accuracy: 0.8775510204081632
```

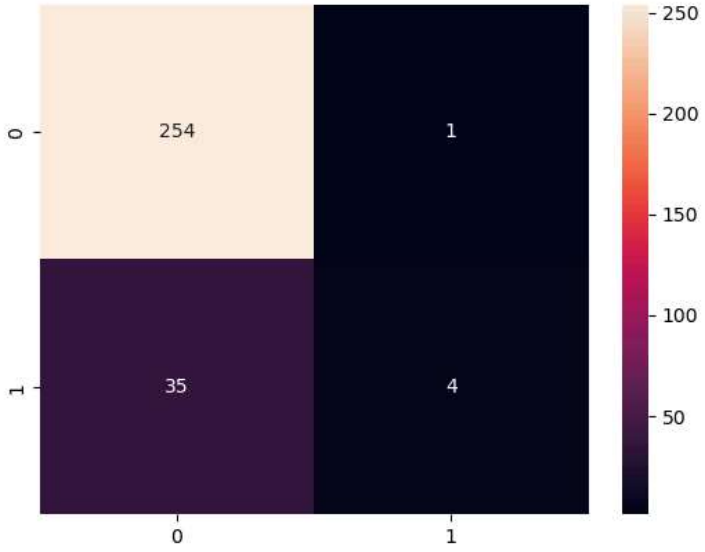
In [37]:

```
# Classification Report

print(classification_report(y_test, y_pred_rf))
```

```
              precision    recall  f1-score
support

           0       0.88      1.00      0.93
255
           1       0.80      0.10      0.18
39

    accuracy                           0.88
294
   macro avg       0.84      0.55      0.56
294
weighted avg       0.87      0.88      0.83
294
```

In [38]:

```
# Confusion Matrix

sns.heatmap(confusion_matrix(y_test, y_pred_rf), annot=True,
plt.show()
```




# Feature Importance

In [41]:

```
importances = rf.feature_importances_

feature_importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': importances
}).sort_values(by='Importance', ascending=False)

print(feature_importance.head(10))
```

```
               Feature  Importance
10      MonthlyIncome    0.074927
44       OverTime_Yes    0.060696
0                 Age    0.049015
11        MonthlyRate    0.044270
17  TotalWorkingYears    0.043394
1           DailyRate    0.042469
4      EmployeeNumber    0.041531
6          HourlyRate    0.040478
20     YearsAtCompany    0.040253
2    DistanceFromHome    0.037208
```

In [ ]:

```

```

In [ ]:

```

```